

Secure Programming Exercise Work

DevSecOps for vulnerable web application

Introduction

This project was implemented within the framework of the course Secure Programming and aimed to create an automated DevSecOps pipeline for the vulnerable Python web application. The application is an open source OWASP training application that consciously contains common security problems.

The project focused on CI/CD automation with the GitHub Actions tool and on improving the security of the application by running security checks on each change with Bandit Scan and Docker Scout. The user does not actually use the program through the interface in this work, but the program operates at the source code level in a CI pipeline environment. The interface is a simple Django-based web application with form fields as well as dynamic views.

In the report the aim is to run GitHub Actions workflow job, scan the web application and analyze the vulnerabilities in the artifacts. When vulnerabilities are analyzed, the aim is to fix them and refactor the web application more secure and prove the positive impact of the DevSecOps workflow.

Vulnerable Web Application

This project utilized a ready-made open-source web application that is intentionally vulnerable. The application is built into the Django web framework and is designed to work for educational purposes for security testing. The application intentionally contains the most common security vulnerabilities based on the OWASP Top 10 listing, such as:

- Injection attacks (e.g. shell or SQL injection)
- Weak authentication
- Hard-coded secrets
- Unsafe handling of information
- Broken access control

The purpose of the application is to provide a realistic test platform to develop and test safe programming methods and DevSecOps practices in practice. This allows security solutions such as static analysis (SAST) and container scanning to be applied to concrete vulnerabilities.

The application runs on Django which provides a pre-configured extensive structure for web development, including user management, database operations, and view management.

Github Actions and Workflow

GitHub Actions is GitHub's built-in CI/CD tool for automating various stages of software development such as testing, checking, building and deployment. It allows the user to configure workflows directly in the version control context. Workflows are defined in a workflow file that is placed in the `.github/workflows/` directory. In this project, the workflow is named as the `main.yml` file. Workflow starts automatically whenever a push is made to the code in GitHub. It consists of two independent jobs, which are SAST scan and image scan which you may see in the pictures 1 and 2 below.

Static analyzing with Bandit performs an SAST check on the Python code using the Bandit tool. It works as follows:

- Load (checkout) project code
- Installs the required Python version and dependencies
- Installs Bandit and updates libraries
- Runs a Bandit scan for the entire project directory and produces a JSON report
- Download the report as an artifact, making it available for later download for analysis, even if the scan fails

```

1  name: CI
2
3  on: [push] # always will be pushed
4
5  jobs:
6    sast_scan: # security sast scanning tool
7      name: Run Bandit Scan
8      runs-on: ubuntu-latest
9
10     steps:
11       - name: Checkout code
12         uses: actions/checkout@v2
13
14       - name: Set up Python #install tools what we need
15         uses: actions/setup-python@v2
16         with:
17           python-version: 3.8
18
19       - name: Install Bandit
20         run: pip install bandit
21
22       - name: Run Bandit Scan
23         run: bandit -ll -ii -r . -f json -o bandit-report.json # everything is scanned and output report implemented
24
25       - name: Upload Artifact # upload the report as an artifact that it is still available after job is executed
26         uses: actions/upload-artifact@v4
27         if: always() # even the previous steps fails, this is executed
28         with:
29           name: bandit-findings
30           path: bandit-report.json
31

```

Picture 1. Workflow for SAST scan with Bandit Tool

This helps to identify, for example, hard-coded passwords, weak encryption algorithms, and injection-dangerous functions already in the development phase.

Docker image building and vulnerability scanning checked that the container image is also safe:

- Retrieving project code
- The Docker environment is initialized

- Logging into the Docker Hub using GitHub's secret variables (no hardcoded IDs)
- Build a Docker image based on the Dockerfile of the project
- Performing a Docker Scout scan looking for critical vulnerabilities in image components
- The report is saved and uploaded as an artifact in SARIF format
- Docker Scout helps ensure that the libraries and packages used by the image do not contain known critical vulnerabilities.

```

32  image_scan:
33    name: Build image and Run Image Scan
34    runs-on: ubuntu-latest
35
36    steps:
37      - name: Checkout code
38        uses: actions/checkout@v3
39
40      - name: Setup Docker # set up docker
41        uses: docker/setup-buildx-action@v3
42
43      - name: Log in to Docker Hub # login to docker using global variables to avoid hard coding user credentials
44        run: echo "${{ secrets.SECURE_PASSWORD }}" | docker login -u "${{ secrets.SECURE_USER }}" --password-stdin
45
46      - name: Build Docker Image # image need to be built before scanning it
47        run: docker build -f Dockerfile -t vulnerableapp:latest .
48
49      - name: Run Docker Scout Scan # run the docker scan: summary of the image and list detailed security vulnerabilities
50        uses: docker/scout-action@v1
51        with:
52          command: quickview,cves
53          image: vulnerableapp:latest
54          only-severities: critical,high
55          sarif-file: scout-report.sarif
56          exit-code: true
57
58      - name: Upload Artifact # upload the report as an artifact that it is still available after job is executed
59        uses: actions/upload-artifact@v4
60        if: always() # even the previous steps fails, this is executed
61        with:
62          name: docker-scout-findings
63          path: scout-report.sarif

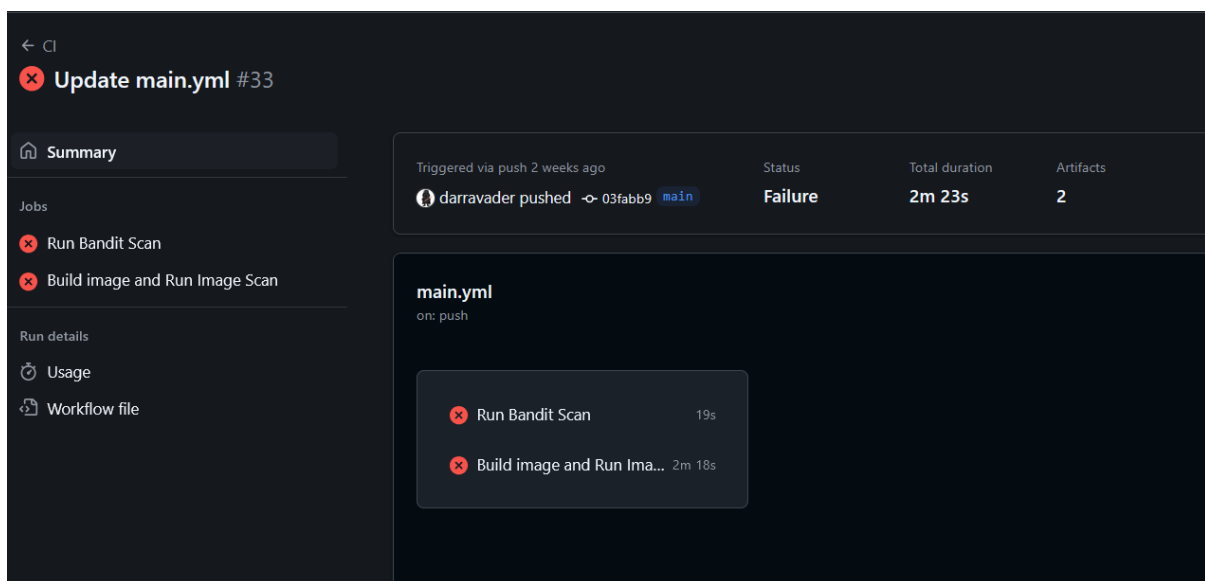
```

Picture 2. Workflow of the image scanning with Docker Scout

This workflow structure ensures that each code change is automatically checked at both the application level (Bandit) and container level (Docker Scout). This is how the DevSecOps principles are applied in practice, where information security is integrated as part of a continuous development process.

Workflow results

Project GitHub Actions workflow fails as expected, because the workflow is implemented to fail with the vulnerabilities is found. This is a desirable and intentional part of security monitoring in the practical DevSecOps model. The job ran for approximately 2 minutes and implemented two artifacts as a result of jobs. The failed job is represented in the picture 3 below.



Picture 3. Github Actions job failure.

Bandit scanning serves as an important component in this, analyzing the Python code for the most known vulnerabilities. The bandit scan was filtered to high and critical severities and it found two high severity vulnerabilities which are represented below in pictures 4 and 5. Both were detected in a code that used the *subprocess.Popen* function with the *shell=True* parameter. Such an implementation can expose the application to shell injection attacks, where the attacker can execute arbitrary commands on the server. This poses a significant security risk, especially if command parameters are formed based on user input. The first vulnerability detected was located in *mitre.py*, line 230. It used *subprocess.Popen(shell=True)* to execute commands. Another similar problem was found in *views.py*, line 423. In both cases, Bandit gave the highest confidence rating, which means that the vulnerability is clearly detectable and based on a well-known code pattern.

```

3 {13}
code : 229 def command_out(command):\n230     process = subprocess.Popen(command, shell=True,
    stdout=subprocess.PIPE, stderr=subprocess.PIPE)\n231     return process.communicate()\n
col_offset : 14
end_col_offset : 99
filename : ./introduction/mitre.py
issue_confidence : HIGH
▶ issue_cwe {2}
issue_severity : HIGH
issue_text : subprocess call with shell=True identified, security issue.
line_number : 230
▶ line_range [1]
more_info : https://bandit.readthedocs.io/en/1.7.10/plugins/b602\_subprocess\_popen\_with\_shell\_equals\_true.html
test_id : B602
test_name : subprocess_popen_with_shell_equals_true

```

Picture 4. Result of Bandit Scan in *mitre.py* file.

```

test_name : subprocess_popen_with_shell_equals_true
7 {13}
code : 422         command,\n423         shell=True,\n424         stdout=subprocess.PIPE, \n425         stderr=subprocess.PIPE)\n426         data = stdout.decode('utf-8')\n427         stdout,
    stderr = process.communicate()\n428         data = stderr.decode('utf-8')\n
col_offset : 26
end_col_offset : 43
filename : ./introduction/views.py
issue_confidence : HIGH
▶ issue_cwe {2}
issue_severity : HIGH
issue_text : subprocess call with shell=True identified, security issue.
line_number : 423
▶ line_range [5]
more_info : https://bandit.readthedocs.io/en/1.7.10/plugins/b602\_subprocess\_popen\_with\_shell\_equals\_true.html
test_id : B602
test_name : subprocess_popen_with_shell_equals_true

```

Picture 5. Result of Bandit Scan in *views.py* file.

The **Docker Scout** tool performed a container image inspection of the application. The original scan contained a large number of vulnerabilities (18 high and 5 critical), so for clarity, the results are filtered to only critical vulnerabilities. Five critically vulnerable packages out of a total of 627 packages were detected as a result of the scan. The result of the scan is represented in picture 6 below.

```

1  ▶ Run docker/scout-action@v1
12 ▶ quickview
28 ▶ cves
29     ✓ SBOM of image already cached, 627 packages indexed
30     ✓ Report written to scout-report.sarif
31
32
33  ## Overview
34
35      | Analyzed Image
36  -----|-----
37  Target | vulnerableapp:latest
38  digest | 2f42aeaa8fe6
39  platform | linux/amd64
40  vulnerabilities | 5C    0H    0M    0L
41  size | 492 MB
42  packages | 627
43
44
```

Picture 6. Result of Docker Scout scan for packages.

One of the detected critical vulnerabilities was related to *PyYAML* version 5.1. There is a problem with this version where malicious code can be hidden in *YAML* files that can run through the application. The recommended solution is to upgrade to *PyYAML* version 5.4 or later.

Another critical vulnerability was related to the current version of *Django* 4.1.7. In this version, input validation can be circumvented in the file upload process, which can lead to security risks. The vulnerability can be fixed by updating *Django* to version 4.1.9 or later.

The third major discovery was related to *Pillow* Library version 9.4.0. In this version, it is possible to execute arbitrary code through the environment parameter, which can compromise the system. The solution is to upgrade *Pillow* to version 10.2.0 or later.

The results of the scans were saved as artifacts to preserve them for later review and documentation. In conclusion, the GitHub Actions workflow identified several critical security flaws in both the application code and its dependencies. Bandit scanning highlighted high-severity vulnerabilities, such as the risky use of the *subprocess.Popen* function, which allows shell injection attacks. The Docker Scout scan, on the other hand, revealed several critical vulnerabilities in third-party libraries such as *PyYAML*,

Django, and *Pillow* packages. These findings highlight the usefulness of the DevSecOps approach, as well as the importance of integrating security tools as part of an automated development process. Without automated SAST and container inspection, these risks might have gone unnoticed, so the results confirm the value of systematic security inspection in software development.

Fixing the vulnerabilities

In the *Mitre.py* file, it was found that using the *subprocess.Popen* function with the *shell=True* parameter exposed the application to shell injection attacks. This meant that attackers could potentially execute malicious commands on the server if the feeds were not filtered accurately enough. Such a vulnerability could lead to serious security issues, such as losing control of the system.

This problem was fixed by changing the use of the *subprocess.Popen* function. Previously, the commands were entered as *shell=True*, but instead of this, the commands were changed to build into a list without the *shell=True* parameter. This ensures that the command is executed directly without the shell environment, greatly reducing the possibility of manipulating commands. This is a safer way to handle external commands, as it prevents the execution of malicious inputs. The refactored code is in picture 7.

```
29 def command_out(command):
30     command_list = ["nmap", ip]
31     process = subprocess.Popen(command_list, stdout=subprocess.PIPE, stderr=subprocess.PIPE)
32     return process.communicate()
33
34 #the command is given directly as a list without the shell=True parameter, which prevents any shell commands from being injected.
35
```

Picture 7. Refactored code of *mitre.py*.

There was another similar vulnerability in the *views.py* file where user inputs (for example domain names) were attached to commands that were run using the *subprocess.Popen* function using the *shell=True* parameter. This allowed attackers to manipulate commands and execute unwanted commands on the server. Such vulnerabilities can lead to, for example, server hijacking or leaking confidential information.

In the fix, the command was changed to a list to be built instead of being merged into a string and executed with the *shell=True* parameter. Commands were run safely without

the use of shell, which prevents the possibility of manipulating commands. This fix ensures that only safe and predictable commands are executed, thus protecting the application from shell injection attacks. The refactored code is in the picture 8.

```
487  def cmd_lab(request):
488      if request.user.is_authenticated:
489          if request.method=="POST":
490              # Get the domain from the POST request and remove 'https://www.'
491              domain = request.POST.get('domain')
492              domain = domain.replace("https://www.", '')
493
494              # Get the operating system from the POST request
495              os = request.POST.get('os')
496              print(os)
497
498              if os == 'win':
499                  # Prepare the command as a list, without shell=True to prevent shell injection
500                  command = ["nslookup", domain]
501              else:
502                  # Prepare the command as a list for UNIX systems
503                  command = ["dig", domain]
504
505              try:
506                  # Execute the command using subprocess without shell=True
507                  process = subprocess.Popen(
508                      command,
509                      stdout=subprocess.PIPE,
510                      stderr=subprocess.PIPE
511                  )
512                  # Capture stdout and stderr
513                  stdout, stderr = process.communicate()
514
515                  # Decode the output to UTF-8
516                  data = stdout.decode('utf-8')
517                  stderr = stderr.decode('utf-8')
518
519                  # Combine stdout and stderr for the output
520                  output = data + stderr
521                  print(data + stderr)
522                  except Exception as e:
523                      # Handle any exceptions, output the error message
524                      output = f"Something went wrong: {e}"
525                      return render(request, 'Lab/CMD/cmd_lab.html', {"output": output})
526
527                  # Print the final output and return it in the response
528                  print(output)
529                  return render(request, 'Lab/CMD/cmd_lab.html', {"output": output})
530              else:
531                  # If method is not POST, just render the initial page
532                  return render(request, 'Lab/CMD/cmd_lab.html')
533          else:
534              # If user is not authenticated, redirect to login page
535              return redirect('login')
```

Picture 8. Refactored code of views.py.

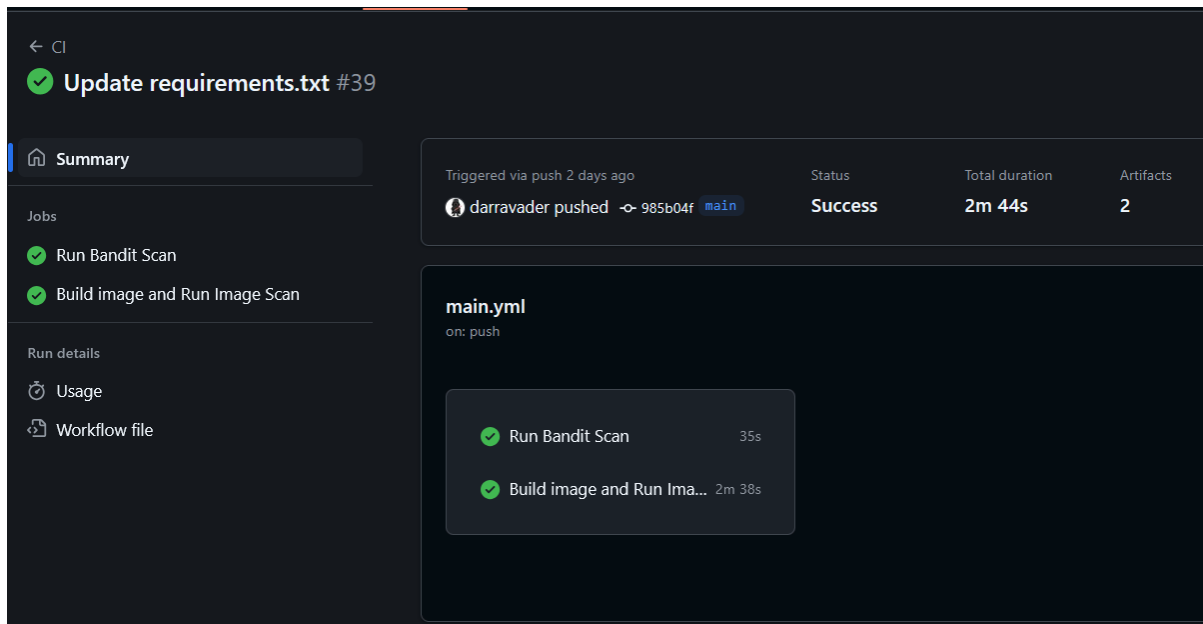
Fixing vulnerabilities with the Docker Scout included installing up-to-date versions of dependencies in the requirements.txt file, which reduced medium and low-severity vulnerabilities, as well as changing the base image from python:3.11.0b1-buster to a more stable and secure python:3.11-slim version with fewer vulnerabilities and dependencies. (Picture 9.)

```
1  argon2-cffi==21.3.0
2  argon2-cffi-bindings==21.2.0
3  asgiref==3.6.0
4  certifi==2022.12.7
5  cffi==1.15.1
6  charset-normalizer==3.0.1
7  cryptography==39.0.1
8  crispy_bootstrap4
9  defusedxml==0.7.1
10 dj-database-url==0.5.0
11 Django==4.1.9
12 django-allauth==0.52.0
13 django-crispy-forms==2.0
14 django-heroku==0.3.1
15 idna==3.4
16 mccabe==0.6.1
17 oauthlib==3.2.2
18 Pillow==10.2.0
19 pycopg2==2.9.3
20 pycodestyle==2.7.0
21 pycparser==2.21
22 pyflakes==2.3.1
23 PyJWT==2.4.0
24 python3-openid==3.2.0
25 pytz==2020.1
26 PyYAML>=5.4
27 requests==2.28.2
28 requests-oauthlib==1.3.1
29 sqlparse==0.3.1
30 urllib3==1.26.9
31 Werkzeug==2.1.2
32 whitenoise==6.2.0
33 zipp==3.8.0
34 Cython==0.29.32
```

Picture 9. *Requirements.txt* file with updated package versions for fixing the issues.

In the workflow file, there was added a step for installing the refactored requirements.txt file. The updated dependencies included verifying the correct version of *PyYAML*, which enabled the use of the finished binary package and prevented translation errors, and updating the version of *Cython* to avoid possible translation errors associated with the use of *Cython*.

As a result, when vulnerabilities were discovered and fixed the GitHub actions workflow job passed, because it did not find any other high or critical severity issues in *Bandit* scan or critical issues in *Docker Scout* (picture 10). As a note, the web application includes several other vulnerabilities in lower severity, which were not taken into account in scanings.



Summary

The goal of the project was to improve the security of the existing open source OWASP Python web application as part of the DevSecOps approach. Based on the initial analysis, several security vulnerabilities were identified in the application, particularly related to command execution and dependency management. The principles of safe software development as well as automated auditing tools were applied to systematically analyse and correct the security flaws of the codebase.

Key repairs were made to the security of command-line calls which were found with *Bandit Scan* SAST tool. *Mitre.py* and *views.py* files used the *subprocess.Popen* function in combination with the *shell=True* parameter, which allowed for the risk of command line injections. This problem was eliminated by building commands as explicit list structures without using the shell environment. With the change, the application no longer interpreted the user's input as part of a complete string on the command line, which prevented malicious commands from being executed on the server.

Another key area was to ensure the security of external dependencies. The *Docker Scout* tool was used to analyze container image vulnerabilities, and the *requirements.txt* file libraries were updated to secure and maintained versions. For example, *PyYAML* and *Cython* libraries were updated to versions that prevent known vulnerabilities and ensure compatibility with new versions of Python. At the same time, Docker's basic image of the unstable *python:3.11.0b1-buster* version was changed to a

lighter and safer *python:3.11-slim image*, which reduced the vulnerabilities included in the container and improved performance.

As a result, the security level of the application improved significantly. Based on static analysis (*Bandit*) and container inspection (*Docker Scout*), many vulnerabilities of high and critical severity were eliminated. The risk level of the application was lowered, and its code base now better meets modern security requirements. As a result of fixes the workflow job was passed due to its severity filters high and critical in *Bandit Scan* and critical in *Docker Scout*.

During the project, the key benefits of the DevSecOps approach were discovered. Automatic scans allow early detection of vulnerabilities as part of continuous development, which speeds up response and improves the quality of the software. DevSecOps makes security part of the normal development process, so it doesn't have to be treated as a separate or an external step. In addition, the use of tools to support developers reduces human error and supports the creation of a secure programming culture.

In conclusion, the implementation of DevSecOps policies helped improve the security of the web application, accelerated the identification and correction of vulnerabilities, and supported the implementation of sustainable and scalable software development.